


29 Sept, 2023
video 22

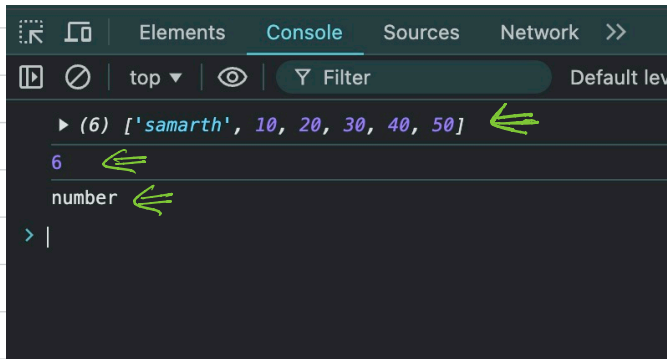
Samarth Sir

25 Oct 2024

Array methods continued...

- ① `.unshift()` $\Rightarrow$ array ke shuru ^{element} me add karta hai. it returns the new length of an array.

```
let arr = [10,20,30,40,50];  
let returnedValue =  
arr.unshift("samarth");  
console.log(arr)  
console.log(returnedValue)  
console.log(typeof(returnedValue))
```



```
▶ (6) ['samarth', 10, 20, 30, 40, 50]  
6  
number  
> |
```

- ② `shift()` $\Rightarrow$ ye array ke aage se pop karayega.

deleted
element
return
karega.

```
let arr = [10,20,30,40,50];  
let returnedValue = arr.shift();  
console.log(arr);  
console.log(returnedValue);
```

```
▶ (4) [20, 30, 40, 50]
10
> |
```

→ array ke elements ko join kar dega.

③ .join() ⇒ sabhi elements ke beech mei augmented stuff rakh dega and string return karega.

```
let arr = [10,20,30,40,50];

let newValue = arr.join('-->');
console.log(newValue);
console.log(typeof(newValue));
let newValue1 = arr.join(' (^_^) ');
console.log(newValue2);
console.log(typeof(newValue2));
```

```
10-->20-->30-->40-->50 app.js:28
string app.js:29
10 (^_^) 20 (^_^) 30 (^_^) 40 (^_^) 50 app.js:31
string app.js:32
10 20 30 40 50 app.js:34
string app.js:35
>
```

```
let arr = [10,20,30,40,50];  
let newValue = arr.join();  
console.log(newValue);  
console.log(typeof(newValue));
```



By default comma lag jayega if there is no argument.

→ array elements join ho gye using comma.

(30:30) Lec 22

④ includes. () → it returns true or false.

```
let arr = [10, 30, 40];  
  
let arr2 = ['arj', 'kri', 'ram'];  
console.log(arr.includes(40));  
  
console.log(arr.includes(20));  
console.log(arr.includes('sam'));  
  
console.log(arr2.includes('sam'));
```

↳ Boolean

includes ye batayega
ki koi element
hai ya nahi.


```
Elements Console Sources Network >>
top Filter Default levels No Issues
true app.js:40
false app.js:42
false app.js:44
false app.js:46
>
```

Q

```
let arr = ['sam', 'samarth', [20], ['sammy', 'boi']];
console.log(arr.includes('sa')); ⇒ l1
console.log(arr.includes('boi')); ⇒ l2
console.log( arr.includes(['sammy', 'boi']) ); ⇒ l3
console.log( arr.includes(["['sammy', 'boi']") ); ⇒ l4
console.log( arr.includes(20) ); ⇒ l5
console.log( arr.includes([20]) ); ⇒ l6
```

```
Elements Console Sources Network >>
top Filter Default levels No Issues
false ⇒ l1 app.js:52
false ⇒ l2 app.js:53
false ⇒ l3 app.js:54
false ⇒ l4 app.js:55
false ⇒ l5 app.js:56
false ⇒ l6 app.js:57
>
```

includes() array ke andar array check nahi karega.

Explanation from Chat gpt:-

In JavaScript, the `includes()` method checks if an element is present in an array, but it only works as expected with primitive values (like numbers or strings) because it uses strict equality (`===`) to compare elements. When it comes to objects or arrays, strict equality checks if they reference the same memory location, not if they have the same content.

Let's break down each `console.log` statement and understand why they're `false`:

1. `console.log(arr.includes('boi'));`

This checks if `'boi'` exists directly in `arr`. However, `'boi'` is nested within a sub-array (`['sammy', 'boi']`). Since `includes()` only checks elements at the top level, it returns `false`.

2. `console.log(arr.includes(['sammy', 'boi']));`

Here, `['sammy', 'boi']` is a new array instance created inline. Although it has the same elements as the sub-array `['sammy', 'boi']` in `arr`, it does not reference the same memory location. Thus, `includes()` returns `false`.

3. `console.log(arr.includes("['sammy', 'boi']"));`

Here, `arr.includes("['sammy', 'boi']")` searches for the string `"['sammy', 'boi']"`, not an array. This string doesn't exist in `arr`, so it returns `false`.

4. `console.log(arr.includes(20));`

This checks if the number `20` exists in `arr`. But `20` is inside a nested array `[20]`, not directly in `arr`. So, `includes()` does not find it at the top level and returns `false`.

5. `console.log(arr.includes([20]));`

Here, `[20]` is a new array instance. Even though it looks like the sub-array `[20]` in `arr`, it references a different memory location. Thus, `includes()` returns `false` because of the strict equality check.



Loops :-

① `for (let i = 1; i <= 10; i++)`

`{`
 `console.log(i);`
`}`

② while

```
let i = 1;
while (i <= 10)
{
    console.log(i);
    i++;
}
```

③ for of $\Rightarrow$ similar to for loop

① for (let i = 0; i < 10; i++) { }

for (let item of arr) { } ②

↓ ↓

iterator(i) jisme iterate karna hai

Syntax of for of loop.

you can compare ① & ②.

we did not write condition & increment in for of.

for of loop hum iterable items par
lagate hai jaise array, linked list, stack.

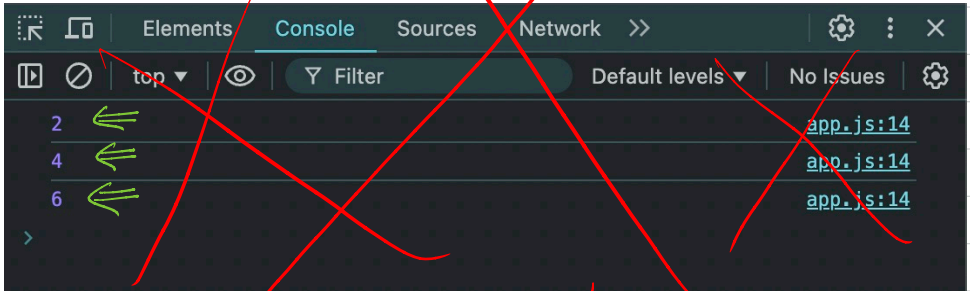
2nd $\Rightarrow$ array ke saath kamesha for of use hoga.

2nd :-

Mathematical operations with undefined results
in NaN in JS.

Q guess the o/p :-

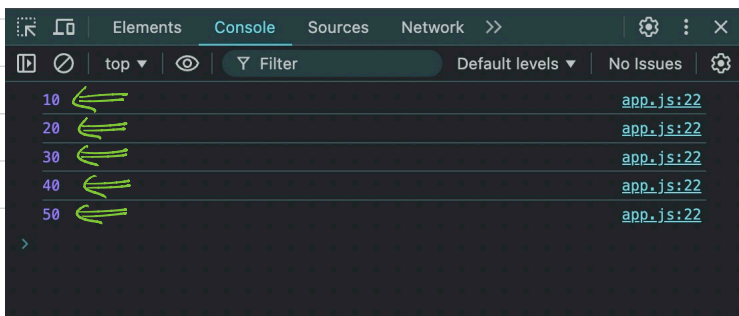
```
let arr= [1, 2, 3];  
for (let i of arr) {  
  console.log(arr[i-1]*2);  
}
```



~~arr [1-1] * 2 $\Rightarrow$ 1 * 2 $\Rightarrow$ 2~~
~~arr [2-1] * 2 $\Rightarrow$ 2 * 2 $\Rightarrow$ 4~~
~~arr [3-1] * 2 $\Rightarrow$ 3 * 2 $\Rightarrow$ 6~~

Q guess the output :-

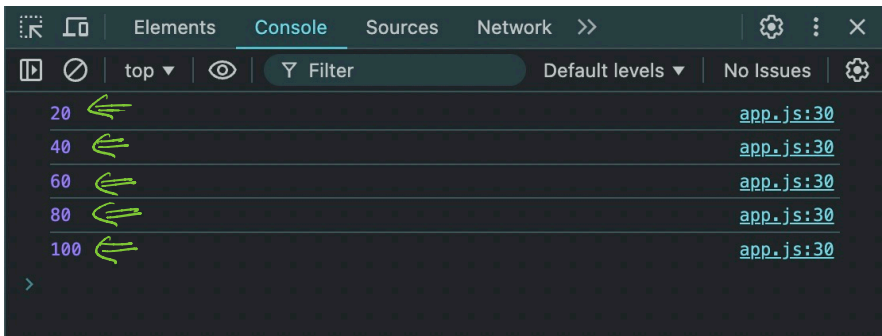
```
let arr = [10, 20, 30, 40, 50];  
for (let i of arr) {  
  console.log(i);  
}
```



रटलौ i is pointing at $arr[i]$, that's why the
op is all array items.

Q guess the output:-

```
let arr = [10, 20, 30, 40, 50];  
for (let i of arr) {  
  console.log(i*2);  
}
```



Loop

- ① $i \rightarrow 10$ i is pointing at $arr[0]$.
- ② $i \rightarrow 20$ " " " " $arr[1]$.
- ③ $i \rightarrow 30$ " " " " $arr[2]$.
- ④ $i \rightarrow 40$ " " " " $arr[3]$.
- ⑤ $i \rightarrow 50$ " " " " $arr[4]$.

Q guess the output :-

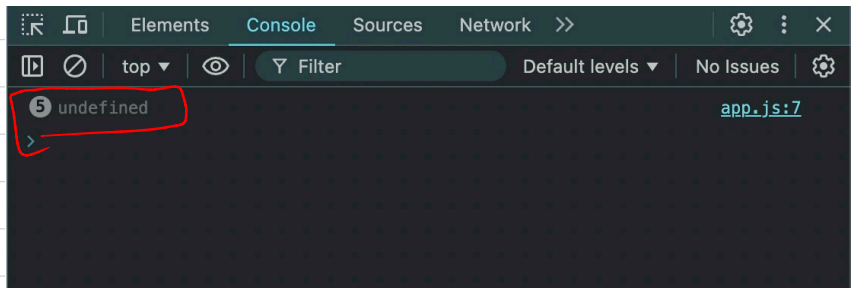
```
let arr = [10, 20, 30, 40, 50];  
for (let i of arr) {  
  console.log(arr[i]);  
}
```

① $i = 10 \Rightarrow \text{arr}[10] \Rightarrow \text{undefined}$

②
⋮

⑤ $i = 50 \Rightarrow \text{arr}[50] \Rightarrow \text{undefined}$

O/p $\Rightarrow$



④ for in :- ye hamesha object ke upar use hoga.

```
let obj = {  
  english : 80,  
  hindi : 99,  
  math : 30  
}
```

```
for(let subject in obj){
```

```
  console.log(subject);
```

→ l1

```
  console.log(obj[key]);
```

→ l2

```
  console.log(obj[subject]);
```

→ l3

```
  console.log(obj.subject);
```

→ l4

```
  console.log(obj.first);
```

→ l5

```
  console.log(obj.math);
```

→ l6

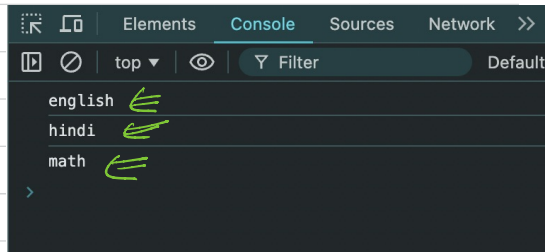
```
}
```

l2 → subject is pointing at key.

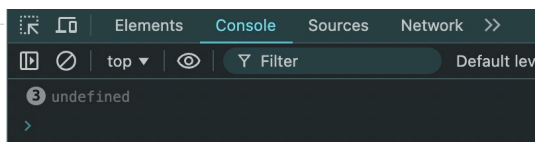
output → english
hindi
math

l2 → size error dea kyunki key variable is not defined.

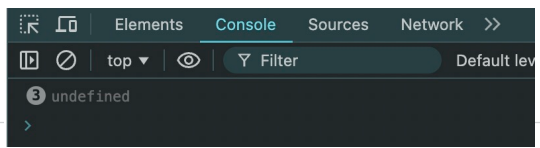
l3 → 80
99 →
30



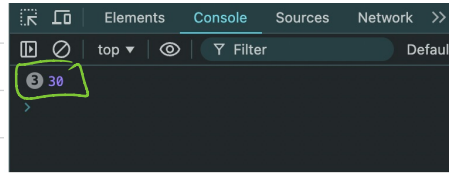
l4 → undefined →



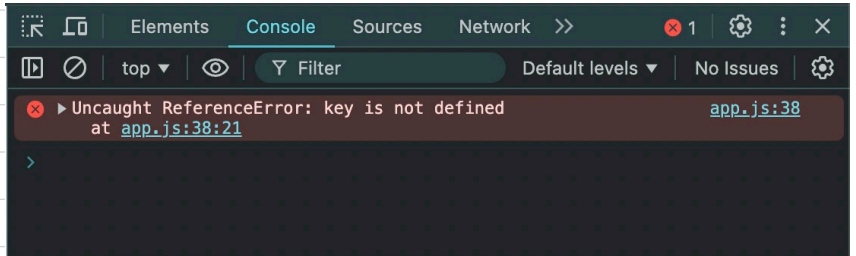
l5 $\Rightarrow$ undefined $\Rightarrow$



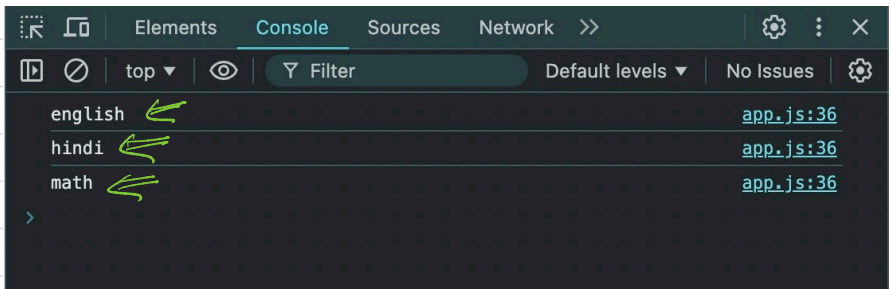
l6 $\Rightarrow$ $\begin{matrix} 30 \\ 30 \\ 30 \end{matrix}$ o/p hoga $\Rightarrow$



l2 $\Rightarrow$



l1 $\Rightarrow$



Loop

- 1 $\rightarrow$ first iteration $\rightarrow$ subject = "english"
- 2 second ,, $\rightarrow$ subject $\rightarrow$ "hindi"
- 3 third ,, $\rightarrow$ subject $\rightarrow$ "math"

Explanation from spt :-

Let's see what each line prints:

- **Line A:** `console.log(subject);`
 - This will print the current key (property name) in `obj`, so it will output each subject name:
- **Line B:** `console.log(obj[key]);`
 - This line will cause an error because `key` is undefined. Since `key` is not declared anywhere in this code, JavaScript will throw a `ReferenceError`.
 - To access properties dynamically, you should use `obj[subject]` instead, which is done in Line C.
- **Line C:** `console.log(obj[subject]);`
 - This line accesses the value of the current property in `obj` using `subject` as the key.
 - For `subject = "english"`, `obj[subject]` is `80`.
 - For `subject = "hindi"`, `obj[subject]` is `99`.
 - For `subject = "math"`, `obj[subject]` is `30`.
 - **Output:**

```
80
99
30
```



- **Line D:** `console.log(obj.subject);`
 - This line will always output `undefined`.
 - `obj.subject` tries to access a property named `"subject"` directly in `obj`, but no such property exists.
- **Line E:** `console.log(obj.first);`
 - This line will also output `undefined`.
 - `obj.first` tries to access a property named `"first"`, which also does not exist in `obj`.
- **Line F:** `console.log(obj.math);`
 - This line will output the value of the `math` property directly, regardless of the current iteration.
 - Since `obj.math` is `30`, it will print `30` in each iteration:

Copy code

```
30
30
30
```

Break & Continue:-

↓
break loop se bahar le aayega

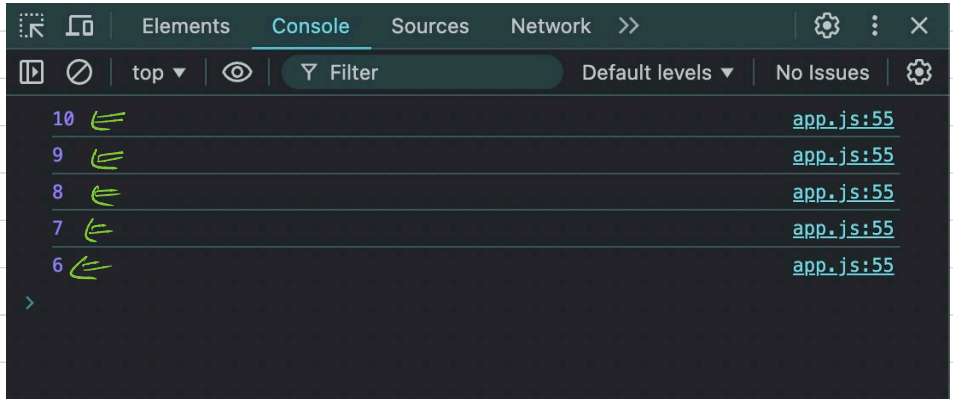
The **break** statement "jumps out" of a loop.

The **continue** statement "jumps over" one iteration in the loop.

↪ reference → w3 schools

Q

```
for(let i = 10 ; i>=0 ; i--){  
    if(i == 5){  
        break;  
    }  
    console.log(i);  
}
```



Elements Console Sources Network >>

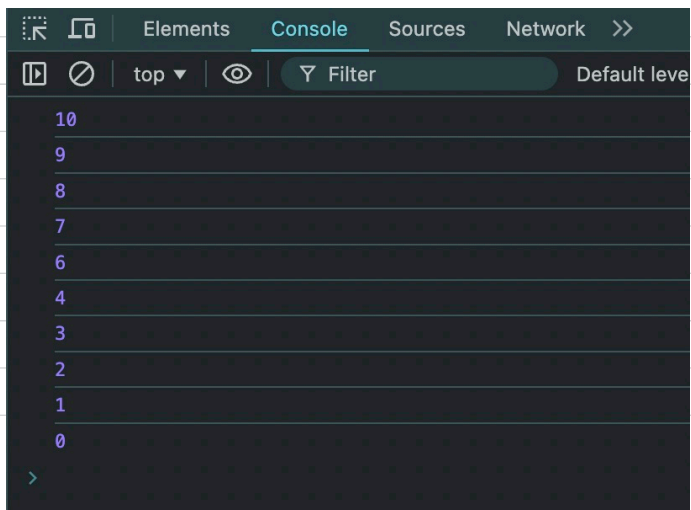
top Filter Default levels No Issues

```
10  
9  
8  
7  
6
```

app.js:55

Q

```
for(let i = 10 ; i>=0 ; i--){  
    if(i == 5){  
        continue;  
    }  
    console.log(i);  
}
```



Elements Console Sources Network >>

top Filter Default levels

```
10  
9  
8  
7  
6  
4  
3  
2  
1  
0
```

app.js:55

Koi particular iteration skip karna ho, then we use continue. Hamne $i=5$ wala skip kar diya isliye 5 console nahi hua.

3 ways declaration:-

Note:- Mai iss note book mei sirf unchijs ko note down karunga jo Shreyance sir ne nahi padaye.

Please refer Shreyance sir notes first:-

File name: [Lec 14 6 July 2024]

JS mei 3 declarations hai

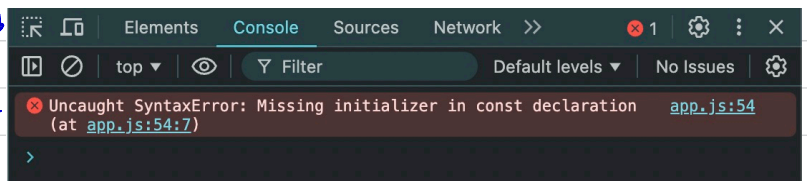
↓
let, var, const.

Q1

Koi value assign nahi hui isliye undefined hoga.

```
const samarth;  
samarth = 100;  
console.log(samarth);
```

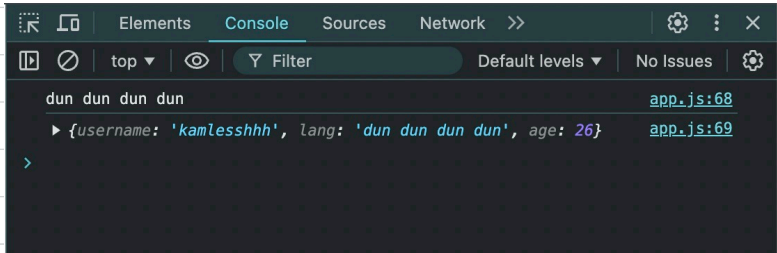
→ not allowed in JS for const



→ But o/p mei error aayega.

Q2

```
const person = {  
  username : "kamlesshhh",  
  lang : "nasheeeee"  
}  
  
person.lang = "dun dun dun dun";  
person.age = 26;  
  
console.log(person.lang)  
console.log(person)
```



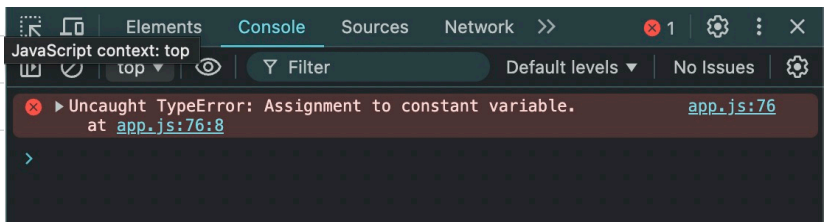
```
dun dun dun dun  
{username: 'kamlesshhh', lang: 'dun dun dun dun', age: 26}
```

Humne object ki key ki value update kar di in const case.

Q3

```
const person = {  
  username : "samarth",  
  lang : "bhaukali"  
};  
person = {  
  username : "sam",  
  lang : "bok"  
};  
  
console.log(person.username)  
console.log(person.lang)
```

error aayega :-



```
Uncaught TypeError: Assignment to constant variable.  
at app.js:76:8
```

Q2 ismei person jis address par point kar raha hai, hamne vo nahi badla, isliye error nahi aaya (in case of const)

Q3 mei person jis address par point kar raha hai, hamne vo change kar diya, isliye error aaya (in case of const).

Explanation:

- Here, `person` is declared as a `const`, which means the reference to the object cannot be changed. However, the contents of the object (its properties) can be modified.
- `person.lang = "dun dun dun dun";` updates the `lang` property of the `person` object. Since you're only modifying the properties of the object, this does not change the reference of `person`.
- `person.age = 26;` adds a new property `age` to the existing `person` object, which is also allowed.
- As a result, this code does not throw an error because it modifies the properties of the object rather than attempting to reassign the `person` variable itself.

→ Q2 explanation ~~for~~ from gpt.

Explanation:

- The keyword `const` makes the variable binding `person` constant, meaning you cannot reassign `person` to a new object or value.
- When you try to do `person = { username: "sam", lang: "bok" };`, JavaScript attempts to reassign the `person` variable to a new object, which is not allowed for `const` variables.
- Therefore, this code will throw an error: `TypeError: Assignment to constant variable.`

→ Q3 explanation from gpt.

Doubt ↓

which means the reference to the object cannot be changed

i didn't get this line.

When we say "the reference to the object cannot be changed" in the context of `const`, we mean that the `const` variable always points to the same object in memory, and you can't make it point to a different object.

Let's break it down further:

1. Understanding "Reference" and "Object"

- A reference is essentially an address or link to where the object is stored in memory.
- When you create an object with `const`, like `const person = { ... };`, `person` is a constant reference to that specific object in memory.

2. What "Cannot Be Changed" Means

- With `const`, you cannot make `person` point to a new object.
 - For example, `person = { name: "New" };` is not allowed because this would mean changing `person` to refer to a new memory location (a different object).

PHASE 1 of JS is OVER.